Note: The line of code *%matplotlib inline* is meant only for Jupyter notebook. So, if you are writing this script on some other IED like Spyder or Visual Studio Code, please remove this line.

Step 4: Plot the linear regression line for the four sub-plots

In this step, we will plot the linear regression line or the 'best fit' line for the four sub-plots. If you look back at the summary statistics for the four data sets, you will notice that the Linear regression line is: y = 3.00 + 0.500x.

So, from the high school maths equation for a straight line (y = mx + c), we know that for all these four data sets, the slope is m = 0.500 and the y-intercept is c = 3.00.

Let us now calculate the slope and intercepts for each of these four 'best fit' lines for each of the four data sets, and see for ourselves if they match the given summary statistics.

For calculating the 'best fit' lines, we will use a Numpy function called *polyfit()*.

The signature to be used is given below:

```
numpy.polyfit(x, y, deg, rcond=None,
full=False, w=None, cov=False)
```

The complete signature of the *numpy.polyfit()* method is available at the link given in the 'References' at the end of the article.

You need to understand that *polyfit()* has an attribute *deg*, which takes the 'degree' of the resulting polynomial to be created. For a linear function, we will take *deg* = 1.

The script for implementing Step 4 is as follows:

```
# ----- STEP 4:- Plot the linear
regression lines for the 4 plots
# Use numpy.polyfit() method. Signature
is at link on next line
# https://numpy.org/doc/stable/
reference/generated/numpy.polyfit.html
import numpy as np

# Use polyfit() to get slope and
intercept
m1, b1 = np.polyfit(df2.X1, df2.Y1, deg
= 1)
print('First dataset->', round(m1, 2),
round(b1, 2))

# m2 and b2 are same as m1 and b1
m2, b2 = np.polyfit(df2.X2, df2.Y2, deg
= 1)
print('Second dataset->', round(m2, 2),
round(b2, 2))

# m3 and b3 are same as m1 and b1
m3, b3 = np.polyfit(df2.X3, df2.Y3, deg
= 1)
print('Third dataset->', round(m3, 2),
round(b3, 2))
# m4 and b4 are same as m1 and b1
m4, b4 = np.polyfit(df2.X4, df2.Y4, deg
= 1)
print('Fourth dataset->', round(m4, 2),
round(b4, 2))
# Infact the slopes and intercepts of
all 4 graphs is same.
# So we can add this best fit line to all
4 graphs as below:-

x = np.array(range(16))
print(x)
ax[0][0].plot(x, m1 * x + b1)
ax[0][1].plot(x, m2 * x + b2)
ax[1][0].plot(x, m3 * x + b3)
ax[1][1].plot(x, m4 * x + b4)
fig
```

Step 5: End note

Just as an additional exercise, let us further explore the *numpy.polyfit()* function. Note that, in general, *numpy.polyfit()* returns the coefficients in descending order of degree, according to the generation equation given below:

$$P(x) = C_n X^n + C_{(n-2)} X^{(n-1)} + - - - - - - + C_0$$

In case you noticed, the second data set in Anscombe's quartet showed a definite non-linear trend. Let us try to use the *numpy.polyfit()* function with degree 2 to see if we can find a 'curve' that better fits this data set than a simple straight line.

The following script does it, and the output shows a near-perfect match:

```
# ----- STEP 5:- Plot the degree 2
polyfit for the second data set
z = np.polyfit(df2.X2, df2.Y2, deg = 2)
print(z)
ax[0][1].plot(x, z[0] * x**2 + z[1] * x
+ z[2])
fig
```

So what did we learn? We learnt that 'summary statistics' may not always give the true picture of a data set. To fully understand the data we must visualise it, and Anscombe's data is proof of the fact that diverse data sets may look 'similar' statistically but have a completely different visual representation. END

References

[1] 'Download data from the Wikipedia article on Anscombe's quartet', *https://en.wikipedia.org/wiki/Anscombe%27s_quartet*

[2] 'The entire script', *https://github.com/ag999git/scripts/blob/main/Anscombe*

[3] 'Complete signature of the function subplots()', *https://matplotlib.org/3.2.1/api/_as_gen/matplotlib.pyplot.subplots.html*

[4] 'The complete signature of numpy.polyfit() method', *https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html*

[5] *https://matplotlib.org/3.1.0/gallery/subplots_axes_and_figures/subplots_demo.html*

[6] 'My GitHub page for the entire script', *https://github.com/ag999git/scripts/blob/main/Anscombe*

By: Anurag Gupta

The author is an engineer by education but a police officer by profession. He is a Python enthusiast who has recently co-authored a book on Python named 'Python Programming: Problem Solving, Packages and Libraries'.